PROJECT REPORT

Intelligent Document Question Answering System using RAG

Generative AI Course Final Project

A Retrieval-Augmented Generation (RAG) based application for answering questions from user-provided documents using document retrieval, semantic embeddings, vector search, and a Large Language Model.

Student Name	Arnav Khulbe
Course	Generative AI
Project Type	Final Project
Platform	Google Colab

GitHub Repository: https://github.com/Arnavkhulbe/intelligent-document-qa-rag

1. Abstract

This project implements an Intelligent Document Question Answering System using Retrieval-Augmented Generation (RAG). The system allows a document to be uploaded, processed into smaller text chunks, converted into semantic vector representations, stored in a vector database, and searched to retrieve relevant information for a user's question. The retrieved context is then supplied to a Large Language Model through Groq so that the final response is grounded in the provided document.

The implementation was developed in a Google Colab notebook using Python and LangChain components. Hugging Face sentence-transformer embeddings are used for semantic representation and ChromaDB is used for vector storage and retrieval. Groq provides the LLM inference layer.

2. Problem Statement

Large documents can contain substantial amounts of information, making it difficult to manually locate the exact information required for a specific question. The objective of this project is to build a system that retrieves relevant portions of a document before generating an answer, thereby improving document-grounded question answering.

3. Objectives

- Build a working Retrieval-Augmented Generation pipeline.
- Allow users to provide a document as the knowledge source.
- Split large document text into manageable overlapping chunks.
- Generate semantic embeddings for document chunks.
- Store and retrieve embeddings using a vector database.
- Use retrieved context to generate document-aware answers with an LLM.
- Implement API-key handling without hard-coding the secret into the notebook.

4. Technologies and Libraries Used

Technology / Library	Purpose
Python	Core programming language
Google Colab	Notebook-based execution environment
LangChain	RAG workflow and LLM integration
LangChain Text Splitters	Document chunking
Hugging Face Sentence Transformers	Semantic text embeddings
ChromaDB	Vector database for similarity search
Groq	Large Language Model inference

5. System Architecture

The system follows the standard RAG pipeline:

1. Document Upload	User provides a document
↓	
2. Document Loading	Document is loaded into LangChain document objects
↓	
3. Text Splitting	Document is divided into overlapping chunks
↓	
4. Embeddings	Each chunk is converted into a vector representation
↓	
5. Vector Database	Vectors are stored in ChromaDB
↓	
6. Retrieval	Relevant chunks are retrieved for the question
↓	
7. Prompt + LLM	Retrieved context and question are sent to Groq LLM
↓	
8. Answer	The system returns a context-aware answer

6. Implementation Workflow

6.1 Document Ingestion

The notebook accepts a document and loads it using LangChain's document loading mechanism. An Alice's Adventures in Wonderland text/Markdown document was used as the knowledge source.

6.2 Text Splitting

RecursiveCharacterTextSplitter divides the loaded document into smaller overlapping chunks. The implemented configuration uses a chunk size of 300 characters and an overlap of 100 characters.

6.3 Embedding Generation

A Hugging Face sentence-transformer embedding model converts each text chunk into a numerical vector, enabling semantic retrieval.

6.4 Vector Storage and Retrieval

The generated embeddings are stored in ChromaDB. A user's question is represented in the embedding space and similarity search returns relevant document chunks.

6.5 Answer Generation

Retrieved chunks are inserted into a prompt together with the user's question. The prompt is passed to the Groq-hosted LLM through LangChain, which generates the answer.

7. Key Implementation Details

- Environment setup: Required Python packages are installed in Google Colab.
- Document upload: A document is uploaded into the Colab runtime.
- Loading: The document is converted into LangChain document objects.
- Chunking: RecursiveCharacterTextSplitter creates 818 chunks for the selected document with the implemented configuration.
- Embedding model: A Hugging Face sentence-transformer model is downloaded and loaded successfully.
- Vector store: ChromaDB stores the generated embeddings.
- LLM integration: LangChain's Groq integration is used for question answering.
- Secure API input: The Groq API key is entered at runtime using getpass rather than being written directly into the notebook.

8. Security and API Key Handling

The project requires an API key for the Groq LLM inference step. The notebook uses Python's getpass mechanism so the key is entered interactively at runtime and is not displayed on screen.

The API key should never be written directly into the notebook source, README, screenshots, or GitHub repository. Anyone running the notebook should enter their own Groq API key when prompted.

9. Testing and Results

The system was tested by loading the selected document, creating document chunks, generating embeddings, storing them in the vector database, retrieving relevant context, and passing that context to the Groq LLM. Questions related to the uploaded document produced answers based on retrieved context.

10. Advantages

- Answers are grounded in a user-provided knowledge source.
- Semantic retrieval can find relevant information even when exact keywords differ.
- The knowledge source can be changed by uploading another document.
- The approach reduces the need to place an entire document directly into an LLM prompt.
- The architecture can be extended to multiple documents and more advanced retrieval strategies.

11. Limitations

- The current implementation is a notebook-based prototype rather than a deployed web application.
- The current workflow requires a valid Groq API key for LLM inference.
- The document upload and query interaction are performed inside Google Colab.
- Answer quality depends on chunking, embeddings, retrieval, and the selected LLM.
- There is no dedicated frontend or production backend in the current version.

12. Future Scope

- Build a web interface where users can upload documents and ask questions.
- Add a backend API using FastAPI or Flask.
- Support PDF, DOCX, TXT, and other document formats.
- Allow multiple documents to be indexed in the same knowledge base.
- Add source citations showing which document chunks were used for each answer.
- Add authentication and secure server-side API-key management.
- Improve retrieval using hybrid search, reranking, or metadata filtering.
- Deploy the application to a cloud platform.

13. Conclusion

The Intelligent Document Question Answering System using RAG demonstrates the core concepts of a modern Retrieval-Augmented Generation application. The project combines document processing, text splitting, semantic embeddings, vector similarity search, and Large Language Model generation into one workflow. The system can retrieve relevant information from a supplied document and use that context to generate an answer.

14. Project Repository

GitHub: https://github.com/Arnavkhulbe/intelligent-document-qa-rag

Primary notebook: Intelligent_Document_QA_RAG.ipynb

End of Report